



Certified Kubernetes Administrator (CKA)

By Cloud Mentor Pro

Updated: 10/04/2026



Mock Exam - 2 (Level Up!)

- *11 Questions*

QUESTION 1 OF 11



Weight: 6

Create a StorageClass named `local-sc` with the following specifications and set it as the default storage class:

- The provisioner should be `kubernetes.io/no-provisioner`
- The volume binding mode should be `WaitForFirstConsumer`
- Volume expansion should be enabled

- ☐ Is the StorageClass local-sc created?
- ☐ Is Provisioner kubernetes.io/no-provisioner used?
- ☐ Is the volume binding set to WaitForFirstConsumer?
- ☐ Is local-sc set to the default storage class?

Solution

Check [kubernetes.io](https://kubernetes.io/docs/concepts/storage/storageclasses/) documentation on StorageClasses

1. Create the StorageClass YAML file:

```
# local-sc.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-sc
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: kubernetes.io/no-provisioner
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

2. Apply the manifest on the specified cluster:

```
kubectl apply -f local-sc.yaml
```

3. Verify the StorageClass

```
kubectl get storageclass
```

You should see `local-sc` marked as `(default)` with the correct provisioner and settings

QUESTION 2 OF 11

Weight: 10

Create a deployment named `logging-deployment` in the namespace `logging-ns` with 1 replica, with the following specifications:

The main container should be named `app-container`, use the image `busybox`, and should run the following command to simulate writing logs:

```
sh -c "while true; do echo 'Log entry' >> /var/log/app/app.log; sleep 5; done"
```

Add a sidecar container named `log-agent` that also uses the `busybox` image and runs the command:

```
tail -f /var/log/app/app.log
```

`log-agent` logs should display the entries logged by the main `app-container`

- ☐ Sidecar displays logs from main container
- ☐ Sidecar container properly configured

Solution

Ensure both containers mount the same emptyDir volume.

2. Create the Deployment YAML - example YAML file below:

```
# logger-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: logging-deployment
  namespace: logging-ns
spec:
  replicas: 1
  selector:
    matchLabels:
      app: logger
  template:
    metadata:
      labels:
        app: logger
    spec:
      volumes:
        - name: log-volume
          emptyDir: {}
      initContainers:
        - name: log-agent
          image: busybox
          command:
            - sh
            - -c
            - "touch /var/log/app/app.log; tail -f /var/log/app/app.log"
      volumeMounts:
        - name: log-volume
          mountPath: /var/log/app
          restartPolicy: Always
      containers:
        - name: app-container
          image: busybox
          command:
            - sh
            - -c
            - "while true; do echo 'Log entry' >> /var/log/app/app.log; sleep 5; done"
          volumeMounts:
            - name: log-volume
              mountPath: /var/log/app
```

3. Apply the deployment:

```
kubectl apply -f logger-deployment.yaml
```

4. Verify that the log-agent container displays log entries:

```
kubectl logs -n logging-ns deployment/logging-deployment -c log-agent
```

You should see repeated `Log entry` lines in the output

QUESTION 3 OF 11

Weight: 10

A Deployment named `webapp-deploy` is running in the `ingress-ns` namespace and is exposed via a Service named `webapp-svc`.

Create an Ingress resource called `webapp-ingress` in the same namespace that will route traffic to the service. The Ingress must:

- Use `pathType: Prefix`
- Route requests sent to path `/` to the backend service
- Forward traffic to port `80` of the service
- Be configured for the host `kodekloud-ingress.app`

Test app availability using the following command:

```
curl -s http://kodekloud-ingress.app/
```

- ☐ Ingress exposed and serving traffic via kodekloud-ingress.app host

Solution

Make sure the path is set correctly and the pathType is Prefix. Use host-based routing

1. Checkout the resources in the `ingress-ns` namespace:

```
kubectl get deployment webapp-deploy -n ingress-ns
kubectl get svc webapp-svc -n ingress-ns
```

2. Create the Ingress YAML file:

```
# webapp-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: webapp-ingress
  namespace: ingress-ns
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: kodekloud-ingress.app
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: webapp-svc
            port:
              number: 80
```

3. Apply the Ingress resource:

```
kubectl apply -f webapp-ingress.yaml
```

4. Test access to the app:

```
curl http://kodekloud-ingress.app/
```

QUESTION 4 OF 11

Weight: 8

Create a new deployment called `nginx-deploy`, with image `nginx:1.16` and `1` replica. Next, upgrade the deployment to version `1.17` using rolling update.

Note: Use the `kubectl apply` command to create or update the deployment.

- ☐ Deployment: nginx-deploy, Image: nginx:1.16
- ☐ Image: nginx:1.16
- ☐ Version upgraded to 1.17

Solution

Explore the `--record` option while creating the deployment while working with the deployment definition file. Then make use of the `kubectl apply` command to create or update the deployment. To create a deployment definition file `nginx-deploy`:

```
kubectl create deployment nginx-deploy --image=nginx:1.16 --dry-run=client -o yaml > deploy.yaml
```

To create a resource from definition file and to record:

```
kubectl apply -f deploy.yaml --record
```

To view the history of deployment `nginx-deploy`:

```
kubectl rollout history deployment nginx-deploy
```

To upgrade the image to next given version:

```
kubectl set image deployment/nginx-deploy nginx=nginx:1.17 --record
```

To view the history of deployment `nginx-deploy`:

```
kubectl rollout history deployment nginx-deploy
```


Weight: 10

Important Note : As of kubernetes 1.19, the CertificateSigningRequest object expects a `signerName`.

- CSR: john-developer Status:Approved
- Role Name: developer, namespace: development, Resource: Pods
- Access: User 'john' has appropriate permissions

Solution manifest file to create a CSR as follows:

To approve this certificate, run: `kubectl certificate approve john-developer`

```
kubectl create role developer --resource=pods --verb=create,list,get,update,delete --namespace=development
```

```
kubectl create rolebinding developer-role-binding --role=developer --user=john --namespace=development
```

```
kubectl auth can-i update pods --as=john --namespace=development
```

QUESTION 6 OF 11

Weight: 10

Create an nginx pod named `nginx-resolver` using the `nginx` image and expose it internally using a `ClusterIP` service called `nginx-resolver-service`.

From within the cluster, verify:

1. DNS resolution of the service name
2. Network reachability of the pod using its IP address

Use the `busybox:1.28` image to perform the lookups.

Save the service DNS lookup output to `/root/CKA/nginx.svc` and the pod IP lookup output to `/root/CKA/nginx.pod`.

- ☐ Pod: nginx-resolver created
- ☐ Service DNS Resolution recorded correctly
- ☐ Pod DNS resolution recorded correctly

Solution

Use the command `kubectl run` and create a nginx pod and busybox pod. Resolve it, nginx service and its pod name from `busybox` pod. To create a pod `nginx-resolver` and expose it internally:

```
kubectl run nginx-resolver --image=nginx
kubectl expose pod nginx-resolver --name=nginx-resolver-service --port=80 --target-port=80 --type=ClusterIP
```

To create a pod `test-nslookup`. Test that you are able to look up the service and pod names from within the cluster:

```
kubectl run test-nslookup --image=busybox:1.28 --rm -it --restart=Never -- nslookup nginx-resolver-service
kubectl run test-nslookup --image=busybox:1.28 --rm -it --restart=Never -- nslookup nginx-resolver-service > /root/CKA/nginx.svc
```

Get the IP of the `nginx-resolver` pod and replace the dots(.) with hyphon(-) which will be used below.

```
kubectl get pod nginx-resolver -o wide
kubectl run test-nslookup --image=busybox:1.28 --rm -it --restart=Never -- nslookup <P-O-D-I-P.default.pod> > /root/CKA/nginx.pod
```


QUESTION 7 OF 11

Weight: 10

Create a static pod on `node01` called `nginx-critical` with the image `nginx`. Make sure that it is recreated/restarted automatically in case of a failure.

For example, use `/etc/kubernetes/manifests` as the static Pod path.

- ☐ Is the static pod configured under `/etc/kubernetes/manifests`?
- ☐ Is pod `nginx-critical-node01` up and running?

Solution

To create a static pod called `nginx-critical` by using below command:

```
kubectl run nginx-critical --image=nginx --dry-run=client -o yaml > static.yaml
```

Copy the contents of this file or use `scp` command to transfer this file from `controlplane` to `node01` node.

```
root@controlplane:~# scp static.yaml node01:/root/
```

To know the IP Address of the `node01` node:

```
root@controlplane:~# kubectl get nodes -o wide
```

```
# Perform SSH
```

```
root@controlplane:~# ssh node01
```

```
OR
```

```
root@controlplane:~# ssh <IP of node01>
```

On `node01` node:

Check if static pod directory is present which is `/etc/kubernetes/manifests`, if it's not present then create it.

```
root@node01:~# mkdir -p /etc/kubernetes/manifests
```

Add that complete path to the `staticPodPath` field in the kubelet `config.yaml` file.

```
root@node01:~# vi /var/lib/kubelet/config.yaml
```

now, move/copy the static.yaml to path `/etc/kubernetes/manifests/`.

```
root@node01:~# cp /root/static.yaml /etc/kubernetes/manifests/
```

Go back to the `controlplane` node and check the status of static pod:

```
root@node01:~# exit
```

```
logout
```

```
root@controlplane:~# kubectl get pods
```

QUESTION 8 OF 11

Weight: 10

Create a Horizontal Pod Autoscaler with name `backend-hpa` for the deployment named `backend-deployment` in the **backend namespace** with the `webapp-hpa.yaml` file located under the root folder.

Ensure that the HPA scales the deployment based on **memory utilization**, maintaining an average memory usage of **65%** across all pods. Configure the HPA with a minimum of 3 replicas and a maximum of 15.

- ☐ Is `backend-hpa` HPA deployed in backend namespace?
- ☐ Is deployment configured for metrics memory utilization?

Solution

Under `/root/` folder you will find a yaml file `webapp-hpa.yaml`. Update the yaml file as per task given.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
  namespace: backend
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend-deployment
  minReplicas: 3
  maxReplicas: 15
  metrics:
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 65
```

Use below command

```
kubectl create -f webapp-hpa.yaml
```

QUESTION 9 OF 11

Weight: 10

Modify the existing `web-gateway` on `cka5673` namespace to handle **HTTPS** traffic on port `443` for `kodekloud.com`, using a **TLS certificate** stored in a secret named `kodekloud-tls`.

- ☐ Is the web gateway configured to listen on the hostname `kodekloud.com`?
- ☐ Is the HTTPS listener configured with the correct TLS certificate?

Solution

Check the configuration of the web-gateway.

```
controlplane ~ → kubectl get gateway web-gateway -n cka5673 -o yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: web-gateway
  namespace: cka5673
  resourceVersion: "21919"
  uid: a1d0e35d-5126-4000-88ec-f440941eed75
spec:
  gatewayClassName: kodekloud
  listeners:
  - allowedRoutes:
      namespaces:
        from: Same
      name: https
      port: 80
      protocol: HTTP
```

The current configuration of the `web-gateway` is incorrect as it is listening on port 80 using the HTTP protocol. To update the `web-gateway` to listen on port 443 with the TLS certificate, use the following manifest:

```
# web-gateway.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: web-gateway
  namespace: cka5673
spec:
  gatewayClassName: kodekloud
  listeners:
  - name: https
    protocol: HTTPS
    port: 443
    hostname: kodekloud.com
    tls:
      certificateRefs:
      - name: kodekloud-tls
```

Apply the configuration as follows:

```
kubectl apply -f web-gateway.yaml
```

QUESTION 10 OF 11

Weight: 10

On the cluster, the team has installed multiple helm charts on a different namespace. By mistake, those deployed resources include one of the vulnerable images called `kodekloud/webapp-color:v1`. Find out the release name and uninstall it.

☐ Is helm release uninstalled?

Solution

In this task, we will use the `helm` commands and `jq` tool. Here are the steps: -

1. Run the `helm ls` command with `-A` option to list the releases deployed on all the namespaces using helm.

```
helm ls -A
```

2. We will use the `jq` tool to extract the image name from the deployments.

```
kubectl get deploy -n <NAMESPACE> <DEPLOYMENT-NAME> -o json | jq -r '.spec.template.spec.containers[].image'
```

Replace `<NAMESPACE>` with the namespace and `<DEPLOYMENT-NAME>` with the deployment name, which we get from the previous commands.

After finding the `kodekloud/webapp-color:v1` image, use the `helm uninstall` to remove the deployed chart that are using this vulnerable image.

```
helm uninstall <RELEASE-NAME> -n <NAMESPACE>
```


QUESTION 11 OF 11

Weight: 6

You are requested to create a NetworkPolicy to allow traffic from frontend apps located in the `frontend` namespace, to backend apps located in the `backend` namespace, but not from the databases in the `databases` namespace. There are three policies available in the `/root` folder. Apply the most restrictive policy from the provided YAML files to achieve the desired result. Do not delete any existing policies.

- ☐ Correct NetworkPolicy applied
- ☐ Incorrect NetworkPolicy is not applied
- ☐ Second incorrect NetworkPolicy is not applied

Solution

Read through all provided NetworkPolicy YAML files carefully. Only one of them restricts traffic from the `databases` namespace while allowing it from `frontend`.

1. On controlplane, Review the contents of the three YAML files:

```
cat /root/net-pol-1.yaml
cat /root/net-pol-2.yaml
cat /root/net-pol-3.yaml
```

2. Understand the differences:

- `net-pol-1.yaml` : Too broad; allows traffic from any namespace with a certain label.
- `net-pol-2.yaml` : Incorrect; explicitly allows both `frontend` and `databases`.
- `net-pol-3.yaml` : Correct; only allows traffic from the `frontend` namespace.

2. Apply the correct policy (`net-pol-3.yaml`):

```
kubectl apply -f /root/net-pol-3.yaml
```

4. Verify it's the only one applied:

```
kubectl get netpol -n backend
```

You should only see `net-policy-3` listed